

L7 – Gradients and Gradient Descent II

Alexander Binder

November 4, 2025

know where to look for

- ⦿ [d2l.ai](#) Appendix A.3 and A.4
- ⦿ <https://pytorch.org/>

Linear and Bilinear mappings are the most common in deep learning. Combinations of these together with activation functions can be dealt with using the chain rule.

- 1 Properties of Gradient Descent
- 2 Stochastic gradient descent
- 3 Why gradient descent is hard in practice?
- 4 Compute the Gradient in PyTorch
- 5 Gradient descent in PyTorch
- 6 Chainrule via matrix multiplication and the Backpropagation algorithm
- 7 Einsum for generalized matrix-matrix multiplication type operations
- 8 Derivatives of Bilinear mappings

Gradient Descent

Basic Algorithm: name: **Gradient Descent**:

- given: step size parameter η , initialize start vector $u^{(0)}$ to a value.
- run while loop, until function value changes (δ_L) drop below a threshold, do at iteration t :
 - $u^{(t+1)} = u^{(t)} - \eta \nabla^{(u)} L(u^{(t)})$
 - compute change of objective to last value: $\delta_L = \|L(u^{(t+1)}) - L(u^{(t)})\|$

questions:

- sensitivity to starting point
- sensitivity to learning rate
- quality of obtained solutions

Lets explore starting point effects:

- in `learnThu8.py` run `tGD2([initvalue])` with $initvalue \in [-4, +4]$ to see the effect of a constant stepsize, but different starting points – see in what minimum you end up.

possible problems of gradient descent I

- we find a local minimum, not the global minimum of a function. Local optimum can be good or bad.
- effects of starting point – for non-convex functions: different starting point leads to possibly different solutions u^* .

Lets explore starting point effects:

- in `learnThu8.py` run `tGD2([initvalue])` with $initvalue \in [-4, +4]$ to see the effect of a constant stepsize, but different starting points – see in what minimum you end up.

solutions with respect to starting point

- **Do not train from scratch** for any practical applications. **ALWAYS fine-tune** a neural net pretrained on a large corpus like ImageNet (later lecture)
- if – in rare cases – one would train from scratch (e.g. a totally new architecture), then careful initialization, and special learning rate treatments are important (later lecture)

see e.g. <https://arxiv.org/abs/1502.01852> for examples how initialization matters for training deep neural networks

training deep neural networks from scratch without care about initialization can result in very bad performance

Lets explore learning rate effects:

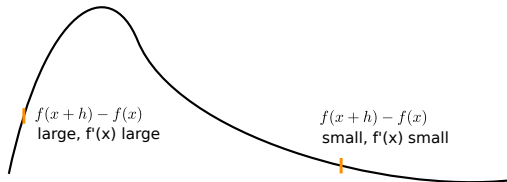
- in `learnThu8.py` run `tGD([stepsize])` to see the effect of different stepsizes.

effects of bad stepsize choices:

- too large $\Rightarrow$ divergence/no solution.
- too small $\Rightarrow$ slow convergence

possible problems of gradient descent

- the size of the update step $u_{t+1} = u_t - \eta \nabla_u L(u_t)$ depends on the norm of the gradient $\|\nabla L(u)\|$, too. So when starting in a steep region ($\|\nabla L(u)\|$ is large), even a small stepsize can lead to divergence.



$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

- one way to deal with the question of how to set the stepsize, is to reduce it over time:

learning rate adjustment schemes / learning rate annealing schemes

In practice: one starts with a learning rate, and decreases it over time, either with a polynomial decrease, or by a factor every N iterations.

polynomial: $\lambda(t) = c_0 \cdot$

$(t+1)^{-\alpha}$, $\alpha > 0$

regular step at each T : $\lambda(t) = c_0 \cdot$

$c^{\lfloor t/T \rfloor}$, $c \in (0, 1)$

in code:

pytorch: `torch.optim.lr_scheduler`

¹

¹What happens if one decreases the learning rate very fast?

- ① Properties of Gradient Descent
- ② Stochastic gradient descent
- ③ Why gradient descent is hard in practice?
- ④ Compute the Gradient in PyTorch
- ⑤ Gradient descent in PyTorch
- ⑥ Chainrule via matrix multiplication and the Backpropagation algorithm
- ⑦ Einsum for generalized matrix-matrix multiplication type operations
- ⑧ Derivatives of Bilinear mappings

Consider setting with an average of **losses over all training samples**, and a predictor which depends on trainable parameters u :

$$\frac{1}{n} \sum_{i=1}^n L(f_u(x_i), y_i)$$

The application of gradient descent to a loss computed over all training samples results in the following algorithm:

$$u_{t+1} = u_t - \eta \nabla_u \left(\frac{1}{n} \sum_{i=1}^n L(f_{u_t}(x_i), y_i) \right)$$

This is called **batch gradient descent** because it uses the set of **all training data samples** to compute the gradient in each step.

The alternative is **stochastic gradient descent** (SGD). This is the default in deep learning.

Stochastic gradient descent for an average of losses

The core idea of **Stochastic gradient descent** is to compute the gradient only over a randomly selected subset of all training samples. After updating the parameters, one draws a new randomly selected subset of all training samples for gradient computation.

SGD in practice

- shuffle/permute your training data $\{z_i = (x_i, y_i), i = 0, \dots, n - 1\}^2$,
- iterate over minibatches , computing the gradient of the loss in each iteration – until all data has been used once
- repeat the two above steps

For example, stochastic gradient descent when starting at index m and using the next k samples uses as update:

$$u_{t+1} = u_t - \eta \nabla_u \frac{1}{k} \sum_{i=m+0}^{m+k-1} L(f_{u_t}(x_i), y_i)$$

²make sure that features x and labels y of one sample (x, y) will stay together!!

stochastic gradient descent for an average of losses

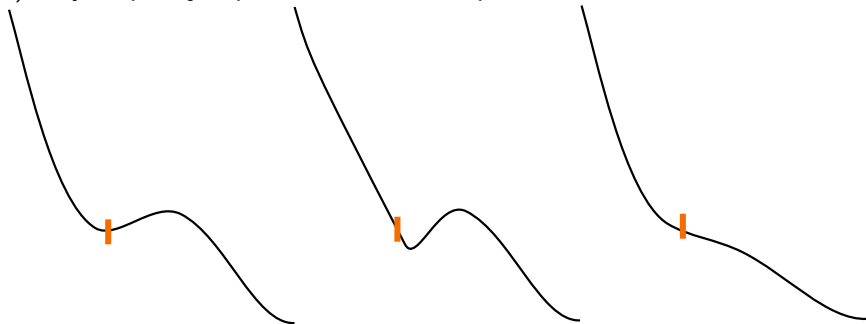
- initialize start vector u_0 as something, choose step size parameter η
- shuffle/permute your training data $\{z_i = (x_i, y_i), i = 0, \dots, n - 1\}$
- until all data has been used once:
 - compute the gradient of the loss for the next k samples (here it starts at an index m)
 - apply it to update the parameters of the mapping f :

$$u_{t+1} = u_t - \eta \nabla_u \frac{1}{k} \sum_{i=m+0}^{m+k-1} L(f_{u_t}(x_i), y_i)$$

- measure loss on validation data . If low enough, stop.
- otherwise repeat from the shuffle step

- ⊙ Full-batch is often too costly to compute a gradient using all samples when its more than tens of thousands
- ⊙ SGD is a noisy, approximated version of the batch gradient (it is based on a random subset, that causes the noise).
 - injecting small noise is one way to prevent overfitting! For deep neural networks SGD can be better than full batch gradient descent in finding *good* local optima.

- An illustration why noise to the loss function (e.g. by randomized sampling of training batches) may help to jump out of bad local optima



Left: a loss surfaces at some point (orange). Middle and Right: changes in the loss surfaces as different training data subsets are used. In the right it allows to jump out of the local minimum.

- 1 Properties of Gradient Descent
- 2 Stochastic gradient descent
- 3 Why gradient descent is hard in practice?
- 4 Compute the Gradient in PyTorch
- 5 Gradient descent in PyTorch
- 6 Chainrule via matrix multiplication and the Backpropagation algorithm
- 7 Einsum for generalized matrix-matrix multiplication type operations
- 8 Derivatives of Bilinear mappings

What makes it hard to use gradient descent in deep neural networks?

- ⊙ one cannot simply stack convolution layers: see for example Fig 1 in <https://arxiv.org/abs/1512.03385>
- ⊙ the problem of vanishing or exploding gradients
 - the graphics on gradient magnitudes in <http://neuralnetworksanddeeplearning.com/chap5.html>

What makes it hard to use gradient descent in deep neural networks?

- vanishing gradients: scale of gradient gets smaller and smaller as neural network architectures become deeper - in particular in layers closer to the input

What makes it hard to use gradient descent in deep neural networks?

- exploding gradients: scale of gradient gets unbounded as neural network architectures become deeper, if learning rates are too high

What makes it hard to use gradient descent in deep neural networks?

- ⊙ imbalanced gradient scales (e.g. Bjorck et al, Understanding Batchnorm <https://arxiv.org/abs/1806.02375>): the scale of gradients has large variation across neurons - this means some neurons have faster updates than others

What makes it hard to use gradient descent in deep neural networks?

- You should know about vanishing/exploding gradients and imbalanced gradient scales
- you should know that gradient descent does not guarantee convergence in every case, due to effects of starting points and learning rate scales

- ① Properties of Gradient Descent
- ② Stochastic gradient descent
- ③ Why gradient descent is hard in practice?
- ④ Compute the Gradient in PyTorch
- ⑤ Gradient descent in PyTorch
- ⑥ Chainrule via matrix multiplication and the Backpropagation algorithm
- ⑦ Einsum for generalized matrix-matrix multiplication type operations
- ⑧ Derivatives of Bilinear mappings

How to, way 1: see `gradtest1.py`

- ⦿ create tensors `a, b, c` explicitly (eg create explicitly using `torch.randn`, or return them from a dataloader)
- ⦿ set `requires_grad=True` on those tensors
 - leaf tensors:
https://docs.pytorch.org/docs/2.7/generated/torch.Tensor.is_leaf.html#torch.Tensor.is_leaf
- ⦿ run operations to obtain a scalar tensor `res` computed from these leaf tensors
- ⦿ `res.backward()` computes the gradients of `res` with respect to all leaf tensors used in its computation
 - inputs to neural networks and weights (due to manual creation, and wrapping as `torch.nn.Parameter`) are usually leaf tensors

How to, way 2: see gradtest2.py

<https://docs.pytorch.org/docs/stable/generated/torch.autograd.grad.html>

- ⦿ still need to set `requires_grad=True` on those tensors
- ⦿ more flexible: can compute grad for multiple outputs ... but
- ⦿ still need scalar outputs (or use `grad_outputs` parameter)

- ① Properties of Gradient Descent
- ② Stochastic gradient descent
- ③ Why gradient descent is hard in practice?
- ④ Compute the Gradient in PyTorch
- ⑤ Gradient descent in PyTorch
- ⑥ Chainrule via matrix multiplication and the Backpropagation algorithm
- ⑦ Einsum for generalized matrix-matrix multiplication type operations
- ⑧ Derivatives of Bilinear mappings

see `fmnist_pytorch_logreg_class.py`

- ⦿ Step 1: wrap trainable tensors in `class torch.nn.Parameter`
 - sets `requires_grad` flag
 - allows to return all parameters of a model using one function
 - together with buffers will be included in the `state_dict` when saving model parameters

```
class onelinear(torch.nn.Module):  
    def __init__(self, dims, numout):  
        super().__init__() #initialize base class  
  
        self.bias=torch.nn.Parameter(data=torch.zeros(numout), requires_grad=True)  
        self.w=torch.nn.Parameter(data=torch.randn( (dims,numout) ), requires_grad=True)
```

see `fmnist_pytorch_logreg_class.py`

- Step 2: hand over all parameters meant to be trained to the optimizer

```
model = onelinear(indims,numcl).to(device)
[...]  
optimizer = optim.SGD(model.parameters(), lr=lr, momentum=0.9)
```

see `fmnist_pytorch_logreg_class.py`

```
for batch_idx, data in enumerate(trainloader):
```

```
    inputs=data[0].to(device)
```

```
    labels=data[1].to(device)
```

```
    outputs = model(inputs)
```

```
    loss = criterion(outputs, labels)
```

```
    optimizer.zero_grad() #reset accumulated gradients
```

```
    loss.backward() #compute new gradients
```

```
    optimizer.step() # apply new gradients to change model parameters
```

see `fmnist_pytorch_logreg_class.py`

```
for batch_idx, data in enumerate(trainloader):  
  
    inputs=data[0].to(device)  
    labels=data[1].to(device)  
  
    outputs = model(inputs)  
    loss = criterion(outputs, labels)  
  
    optimizer.zero_grad() #reset accumulated gradients  
    loss.backward() #compute new gradients  
    optimizer.step() # apply new gradients to change model parameters
```

- step 3: compute model prediction, and loss from the prediction
- Step 4: `optimizer.zero_grad()` to erase all previously computed gradients in all instances of `class torch.nn.Parameter` which were passed to the optimizer
- Step 5: `loss.backward()` to get new gradients in the `.grad` fields of all instances of `class torch.nn.Parameter`
- Step 6: `optimizer.step()` to Perform a variant of Gradient descent (using the `.grad` fields of `class torch.nn.Parameter` to update the parameter values)

- ① Properties of Gradient Descent
- ② Stochastic gradient descent
- ③ Why gradient descent is hard in practice?
- ④ Compute the Gradient in PyTorch
- ⑤ Gradient descent in PyTorch
- ⑥ Chainrule via matrix multiplication and the Backpropagation algorithm
- ⑦ Einsum for generalized matrix-matrix multiplication type operations
- ⑧ Derivatives of Bilinear mappings

We had for two mappings, that the chainrule tells us that we have to concatenate the three linearizations:

$$D(\mathbf{g} \circ \mathbf{h})|_{\mathbf{x}}[\mathbf{v}] = D\mathbf{g}|_{\mathbf{h}(\mathbf{x})}[D\mathbf{h}|_{\mathbf{x}}[\mathbf{v}]]$$

We had for three mappings, that the chainrule tells us that we have to concatenate the three linearizations:

$$D(\mathbf{f} \circ \mathbf{g} \circ \mathbf{h})|_{\mathbf{x}}[\mathbf{v}] = D\mathbf{f}|_{\mathbf{g} \circ \mathbf{h}(\mathbf{x})}[D\mathbf{g}|_{\mathbf{h}(\mathbf{x})}[D\mathbf{h}|_{\mathbf{x}}[\mathbf{v}]]]$$

We know for a real-valued function $f(x) \in \mathbb{R}$ that $Df|_{\mathbf{x}}[\mathbf{v}] = \nabla f|_{\mathbf{x}} \cdot \mathbf{v}$

- ◉ We know for a real-valued function $f(x) \in \mathbb{R}$ that $Df|_x[v] = \nabla f|_x \cdot v$ if we write $\nabla f|_x$ as a column vector, then

$$\nabla f|_x = \begin{pmatrix} \frac{\partial f}{\partial x_0}|_x \\ \frac{\partial f}{\partial x_1}|_x \\ \vdots \\ \frac{\partial f}{\partial x_{k-1}}|_x \end{pmatrix}$$

when writing this as matrix-multiplication, we obtain:

$$Df|_x[v] = [\nabla f|_x]^\top v$$

- ◉ this extends naturally to vector-valued outputs $f(x) = (f_0(x), \dots, f_{r-1}(x))$ and the corresponding Jacobi-matrix

$$Jf|_x = (\nabla f_0|_x, \dots, \nabla f_{r-1}|_x)$$

- this extends naturally to vector-valued outputs $f(x) = (f_0(x), \dots, f_{r-1}(x))$ and the corresponding Jacobi-matrix

$$\begin{aligned}
 Jf|_x &= (\nabla f_0|_x, \dots, \nabla f_{r-1}|_x) \\
 &= \begin{pmatrix} \frac{\partial f_0}{\partial x_0}|_x & \frac{\partial f_1}{\partial x_0}|_x & \frac{\partial f_2}{\partial x_0}|_x & \cdots & \frac{\partial f_{r-1}}{\partial x_0}|_x \\ \frac{\partial f_0}{\partial x_1}|_x & \frac{\partial f_1}{\partial x_1}|_x & \frac{\partial f_2}{\partial x_1}|_x & \cdots & \frac{\partial f_{r-1}}{\partial x_1}|_x \\ \frac{\partial f_0}{\partial x_2}|_x & \frac{\partial f_1}{\partial x_2}|_x & \frac{\partial f_2}{\partial x_2}|_x & \cdots & \frac{\partial f_{r-1}}{\partial x_2}|_x \\ \vdots & \vdots & \ddots & \cdots & \vdots \\ \frac{\partial f_0}{\partial x_{k-1}}|_x & \frac{\partial f_1}{\partial x_{k-1}}|_x & \frac{\partial f_2}{\partial x_{k-1}}|_x & \cdots & \frac{\partial f_{r-1}}{\partial x_{k-1}}|_x \end{pmatrix} \\
 Df|_x[\mathbf{v}] &= [Jf|_x]^\top \mathbf{v}
 \end{aligned}$$

- this can be directly applied to the chainrule

$$\begin{aligned}
 Jf|_{\mathbf{x}} &= (\nabla f_0|_{\mathbf{x}}, \dots, \nabla f_{r-1}|_{\mathbf{x}}) \\
 &= \begin{pmatrix} \frac{\partial f_0}{\partial x_0}|_{\mathbf{x}} & \frac{\partial f_1}{\partial x_0}|_{\mathbf{x}} & \frac{\partial f_2}{\partial x_0}|_{\mathbf{x}} & \cdots & \frac{\partial f_{r-1}}{\partial x_0}|_{\mathbf{x}} \\ \frac{\partial f_0}{\partial x_1}|_{\mathbf{x}} & \frac{\partial f_1}{\partial x_1}|_{\mathbf{x}} & \frac{\partial f_2}{\partial x_1}|_{\mathbf{x}} & \cdots & \frac{\partial f_{r-1}}{\partial x_1}|_{\mathbf{x}} \\ \frac{\partial f_0}{\partial x_2}|_{\mathbf{x}} & \frac{\partial f_1}{\partial x_2}|_{\mathbf{x}} & \frac{\partial f_2}{\partial x_2}|_{\mathbf{x}} & \cdots & \frac{\partial f_{r-1}}{\partial x_2}|_{\mathbf{x}} \\ \vdots & \vdots & \ddots & \cdots & \vdots \\ \frac{\partial f_0}{\partial x_{k-1}}|_{\mathbf{x}} & \frac{\partial f_1}{\partial x_{k-1}}|_{\mathbf{x}} & \frac{\partial f_2}{\partial x_{k-1}}|_{\mathbf{x}} & \cdots & \frac{\partial f_{r-1}}{\partial x_{k-1}}|_{\mathbf{x}} \end{pmatrix} \\
 Df|_{\mathbf{x}}[\mathbf{v}] &= [Jf|_{\mathbf{x}}]^\top \mathbf{v}
 \end{aligned}$$

- ◉ this can be directly applied to the chainrule

$$\begin{aligned}
 D(\mathbf{f} \circ \mathbf{g} \circ \mathbf{h})|_{\mathbf{x}}[\mathbf{v}] &= D\mathbf{f}|_{\mathbf{g} \circ \mathbf{h}(\mathbf{x})}[D\mathbf{g}|_{\mathbf{h}(\mathbf{x})}[D\mathbf{h}|_{\mathbf{x}}[\mathbf{v}]]] \\
 &= [J\mathbf{f}|_{\mathbf{g} \circ \mathbf{h}(\mathbf{x})}]^\top [J\mathbf{g}|_{\mathbf{h}(\mathbf{x})}]^\top [J\mathbf{h}|_{\mathbf{x}}]^\top \mathbf{v}
 \end{aligned}$$

- ◉ insight: we can implement chainrule as a sequence of matrix multiplications of Jacobians !
(directional derivatives are linear operations, matrix multiplications are also linear operations)

key insight

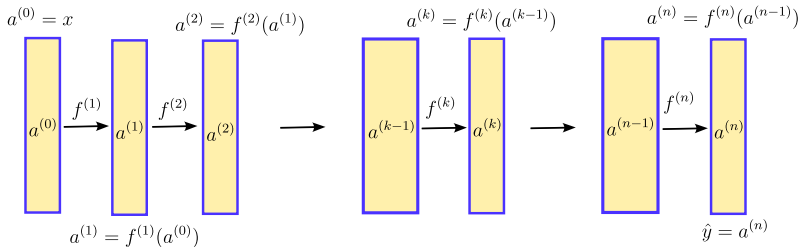
We can implement chainrule for a sequence of functions

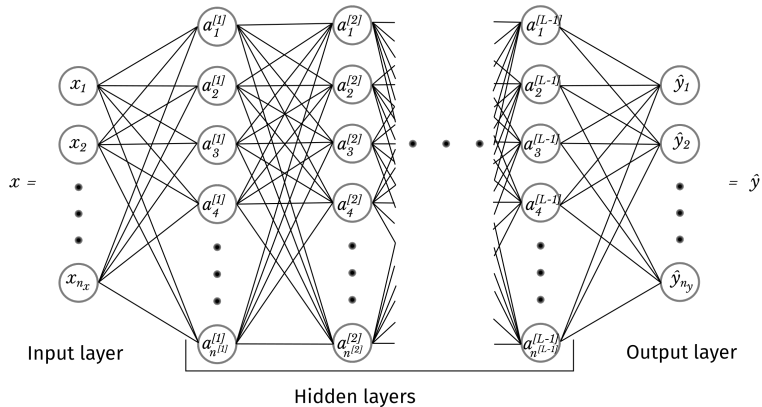
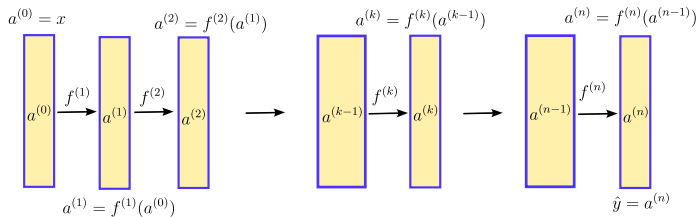
$$f^{(n)} \circ f^{(n-1)} \circ \dots \circ f^{(2)} \circ f^{(1)}$$

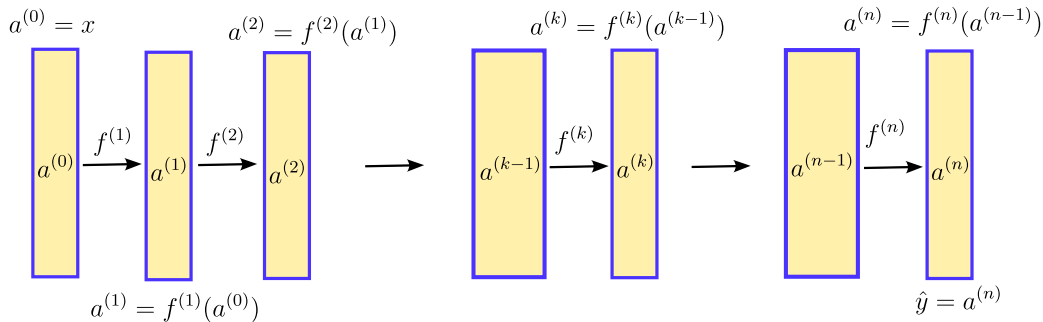
as a sequence of matrix multiplications of transposed Jacobian matrices

$$[Jf^{(n)}]^\top [Jf^{(n-1)}]^\top \dots [Jf^{(2)}]^\top [Jf^{(1)}]^\top$$

- we will apply it to the chain of function layers in a neural network!





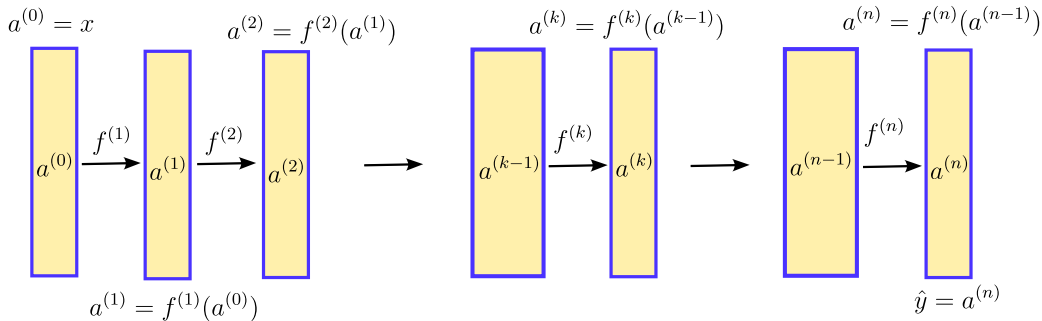


The whole neural network is a chain of functions

$$\hat{y} = g(x) = f^{(n)} f^{(n-1)} f^{(n-2)} \dots f^{(2)} f^{(1)}(x)$$

in particular

$$\begin{aligned} x &= a^{(0)} \\ f^{(k)}(a^{(k-1)}) &= a^{(k)} \\ f^{(n)}(a^{(n-1)}) &= a^{(n)} = \hat{y} \end{aligned}$$



The whole neural network is a chain of functions (corresponding to layers)

$$\hat{y} = g(x) = f^{(n)} f^{(n-1)} f^{(n-2)}$$

$$\dots f^{(2)} f^{(1)}(x)$$

$$Dg(x)[v] = [Jf^{(n)}]^\top [Jf^{(n-1)}]^\top [Jf^{(n-2)}]^\top$$

$$\dots [Jf^{(2)}]^\top [Jf^{(1)}]^\top v$$

Its derivative is a chain of matrix multiplications of corresponding Jacobians for the respective layers

- how to use this to run backpropagation ?

The whole neural network is a chain of functions (corresponding to layers). We make the dependence on parameters $w^{(n)}$ of layer function more explicit:

$$\hat{y} = f_{w^{(n)}}^{(n)} f_{w^{(n-1)}}^{(n-1)} f_{w^{(n-2)}}^{(n-2)} \dots f_{w^{(2)}}^{(2)} f_{w^{(1)}}^{(1)}(x)$$

For every layer function $f^{(k)}$ we can compute a Jacobian (cf. derivative)

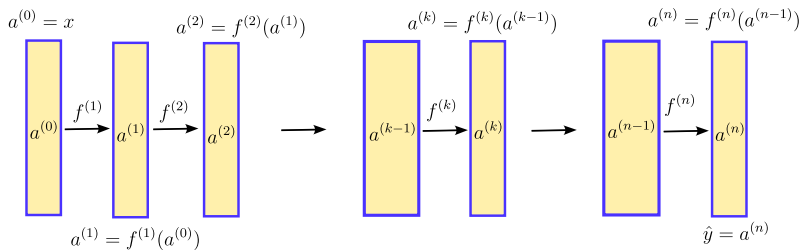
$$[J^{(a^{(k-1)})} f^{(k)}]^\top$$

wrt. to the layer inputs $a^{(k-1)}$ and a Jacobian

$$[J^{(w^{(k)})} f^{(k)}]^\top$$

wrt. to the layer parameters $w^{(k)}$.

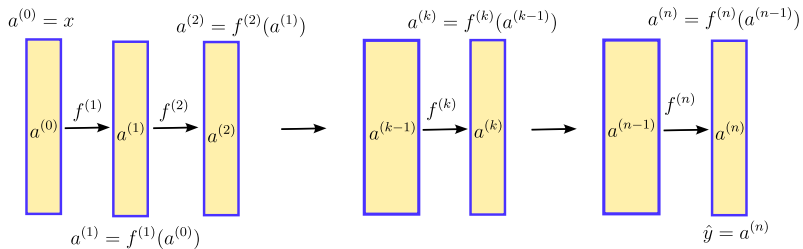
- for updating the network parameters w of the network function g in x we need all derivatives $J^{(w^k)} g$ for all $w^k, k = 1, \dots, n$



The derivatives for the top-most layer can be computed directly:

$$\hat{y} = g(x) = f^{(n)} f^{(n-1)} f^{(n-2)} \dots f^{(2)} f^{(1)}(x)$$

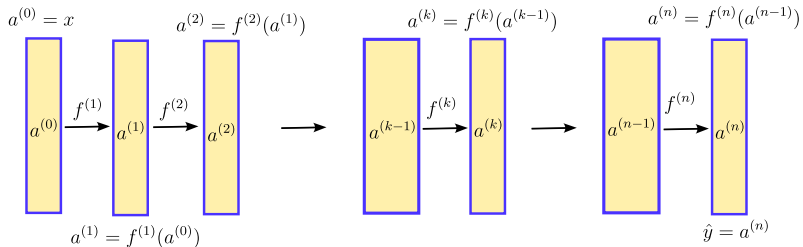
$$[J^{(w^n)} g]^\top = [J^{(w^n)} f^{(n)}]^\top$$



The derivatives for the second layer from the top:

$$g(x) = f^{(n)} f^{(n-1)} f^{(n-2)} \dots f^{(2)} f^{(1)}(x)$$

$$[J^{(w^{n-1})} g]^\top = ???$$

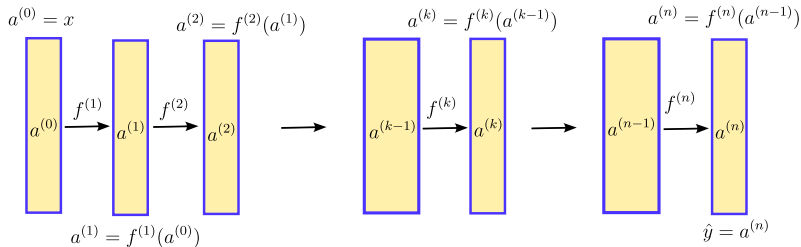


The derivatives for the second layer from the top:

$$g(x) = f^{(n)} f^{(n-1)} f^{(n-2)} \dots f^{(2)} f^{(1)}(x)$$

note: $a^{n-1} = f_{w^{n-1}}^{(n-1)}(a^{(n-2)})$

and: $g(x) = f^{(n)}(a^{n-1})$



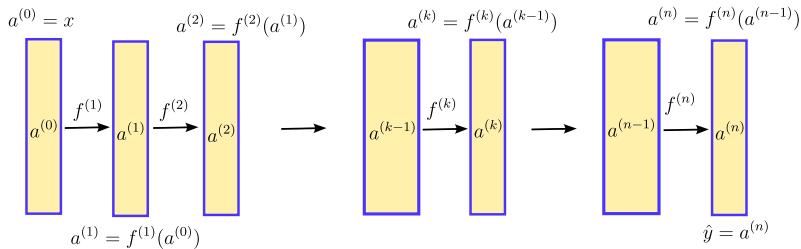
The derivatives for the second layer from the top:

$$g(x) = f^{(n)} f^{(n-1)} f^{(n-2)} \dots f^{(2)} f^{(1)}(x)$$

$$\text{note: } a^{n-1} = f_{w^{n-1}}^{(n-1)}(a^{(n-2)})$$

$$\text{and: } g(x) = f^{(n)}(a^{n-1})$$

$$\Rightarrow [J^{(w^{n-1})} g]^\top = [J^{(a^{n-1})} f^{(n)}]^\top [J^{(w^{n-1})} f^{(n-1)}]^\top$$

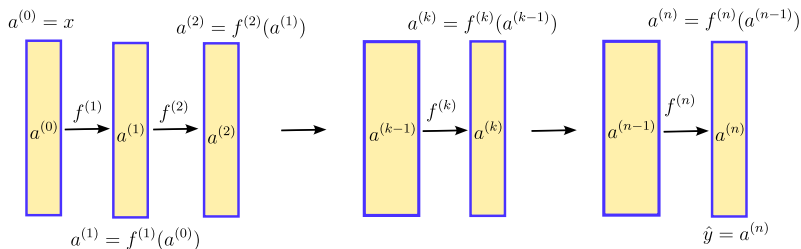


The derivatives for the third layer from the top:

$$g(x) = f^{(n)} f^{(n-1)} f^{(n-2)} \dots f^{(2)} f^{(1)}(x)$$

$$[J(w^{n-1})g]^\top = [J(a^{n-1})f^{(n)}]^\top [J(w^{n-1})f^{(n-1)}]^\top$$

$$[J(w^{n-2})g]^\top = [J(a^{n-1})f^{(n)}]^\top [J(a^{n-2})f^{(n-1)}]^\top [J(w^{n-2})f^{(n-2)}]^\top$$



The derivatives for few more layers from the top:

$$\begin{aligned}
 g(x) &= f^{(n)} f^{(n-1)} f^{(n-2)} \dots f^{(2)} f^{(1)}(x) \\
 [J^{(w^{n-1})} g]^\top &= [J^{(a^{n-1})} f^{(n)}]^\top [J^{(w^{n-1})} f^{(n-1)}]^\top \\
 [J^{(w^{n-2})} g]^\top &= [J^{(a^{n-1})} f^{(n)}]^\top [J^{(a^{n-2})} f^{(n-1)}]^\top [J^{(w^{n-2})} f^{(n-2)}]^\top \\
 [J^{(w^{n-3})} g]^\top &= [J^{(a^{n-1})} f^{(n)}]^\top [J^{(a^{n-2})} f^{(n-1)}]^\top [J^{(a^{n-3})} f^{(n-2)}]^\top [J^{(w^{n-3})} f^{(n-3)}]^\top \\
 [J^{(w^{n-4})} g]^\top &= [J^{(a^{n-1})} f^{(n)}]^\top [J^{(a^{n-2})} f^{(n-1)}]^\top [J^{(a^{n-3})} f^{(n-2)}]^\top [J^{(a^{n-4})} f^{(n-3)}]^\top [J^{(w^{n-4})} f^{(n-4)}]^\top
 \end{aligned}$$

This can be computed recursively!!

The derivatives for few more layers from the top:

$$\begin{aligned}
 g(x) &= f^{(n)} f^{(n-1)} f^{(n-2)} \dots f^{(2)} f^{(1)}(x) \\
 [J^{(w^{n-1})} g]^\top &= [J^{(a^{n-1})} f^{(n)}]^\top [J^{(w^{n-1})} f^{(n-1)}]^\top \\
 [J^{(w^{n-2})} g]^\top &= [J^{(a^{n-1})} f^{(n)}]^\top [J^{(a^{n-2})} f^{(n-1)}]^\top [J^{(w^{n-2})} f^{(n-2)}]^\top \\
 [J^{(w^{n-3})} g]^\top &= [J^{(a^{n-1})} f^{(n)}]^\top [J^{(a^{n-2})} f^{(n-1)}]^\top [J^{(a^{n-3})} f^{(n-2)}]^\top [J^{(w^{n-3})} f^{(n-3)}]^\top \\
 [J^{(w^{n-4})} g]^\top &= [J^{(a^{n-1})} f^{(n)}]^\top [J^{(a^{n-2})} f^{(n-1)}]^\top [J^{(a^{n-4})} f^{(n-2)}]^\top [J^{(a^{n-4})} f^{(n-3)}]^\top [J^{(w^{n-4})} f^{(n-4)}]^\top
 \end{aligned}$$

This can be computed recursively!! The start:

$$\begin{aligned}
 g(x) &= f^{(n)} f^{(n-1)} f^{(n-2)} \dots f^{(2)} f^{(1)}(x) \\
 [J^{(w^n)} g]^\top &= [J^{(w^n)} f^{(n)}]^\top \\
 BP^{(n)} &:= [J^{(a^{n-1})} f^{(n)}]^\top
 \end{aligned}$$

The derivatives for few more layers from the top:

$$\begin{aligned}
 g(x) &= f^{(n)} f^{(n-1)} f^{(n-2)} \dots f^{(2)} f^{(1)}(x) \\
 [J^{(w^{n-1})} g]^\top &= [J^{(a^{n-1})} f^{(n)}]^\top [J^{(w^{n-1})} f^{(n-1)}]^\top \\
 [J^{(w^{n-2})} g]^\top &= [J^{(a^{n-1})} f^{(n)}]^\top [J^{(a^{n-2})} f^{(n-1)}]^\top [J^{(w^{n-2})} f^{(n-2)}]^\top \\
 [J^{(w^{n-3})} g]^\top &= [J^{(a^{n-1})} f^{(n)}]^\top [J^{(a^{n-2})} f^{(n-1)}]^\top [J^{(a^{n-3})} f^{(n-2)}]^\top [J^{(w^{n-3})} f^{(n-3)}]^\top \\
 [J^{(w^{n-4})} g]^\top &= [J^{(a^{n-1})} f^{(n)}]^\top [J^{(a^{n-2})} f^{(n-1)}]^\top [J^{(a^{n-4})} f^{(n-2)}]^\top [J^{(a^{n-4})} f^{(n-3)}]^\top [J^{(w^{n-4})} f^{(n-4)}]^\top
 \end{aligned}$$

This can be computed recursively!! The continuation:

$$\begin{aligned}
 g(x) &= f^{(n)} f^{(n-1)} f^{(n-2)} \dots f^{(2)} f^{(1)}(x) \\
 BP^{(n)} &:= [J^{(a^{n-1})} f^{(n)}]^\top \\
 [J^{(w^{n-1})} g]^\top &= BP^{(n)} [J^{(w^{n-1})} f^{(n-1)}]^\top \\
 BP^{(n-1)} &:= BP^{(n)} [J^{(a^{n-2})} f^{(n-1)}]^\top \\
 [J^{(w^{n-2})} g]^\top &= BP^{(n-1)} [J^{(w^{n-2})} f^{(n-2)}]^\top
 \end{aligned}$$

The derivatives for few more layers from the top:

$$\begin{aligned}
 g(x) &= f^{(n)} f^{(n-1)} f^{(n-2)} \dots f^{(2)} f^{(1)}(x) \\
 [J^{(w^{n-1})} g]^\top &= [J^{(a^{n-1})} f^{(n)}]^\top [J^{(w^{n-1})} f^{(n-1)}]^\top \\
 [J^{(w^{n-2})} g]^\top &= [J^{(a^{n-1})} f^{(n)}]^\top [J^{(a^{n-2})} f^{(n-1)}]^\top [J^{(w^{n-2})} f^{(n-2)}]^\top \\
 [J^{(w^{n-3})} g]^\top &= [J^{(a^{n-1})} f^{(n)}]^\top [J^{(a^{n-2})} f^{(n-1)}]^\top [J^{(a^{n-3})} f^{(n-2)}]^\top [J^{(w^{n-3})} f^{(n-3)}]^\top \\
 [J^{(w^{n-4})} g]^\top &= [J^{(a^{n-1})} f^{(n)}]^\top [J^{(a^{n-2})} f^{(n-1)}]^\top [J^{(a^{n-4})} f^{(n-2)}]^\top [J^{(a^{n-4})} f^{(n-3)}]^\top [J^{(w^{n-4})} f^{(n-4)}]^\top
 \end{aligned}$$

This can be computed recursively!! The whole:

$$\begin{aligned}
 g(x) &= f^{(n)} f^{(n-1)} f^{(n-2)} \dots f^{(2)} f^{(1)}(x) \\
 [J^{(w^n)} g]^\top &= [J^{(w^n)} f^{(n)}]^\top \text{ init} \\
 BP^{(n)} &:= [J^{(a^{n-1})} f^{(n)}]^\top \text{ init} \\
 BP^{(k-1)} &:= BP^{(k)} [J^{(a^{k-2})} f^{(k-1)}]^\top \\
 [J^{(w^{k-2})} g]^\top &= BP^{(k-1)} [J^{(w^{k-2})} f^{(k-2)}]^\top
 \end{aligned}$$

This can be computed recursively!! The whole:

$$\begin{aligned}
 g(x) &= f^{(n)} f^{(n-1)} f^{(n-2)} \dots f^{(2)} f^{(1)}(x) \\
 [J^{(w^n)} g]^\top &= [J^{(w^n)} f^{(n)}]^\top \text{ init} \\
 BP^{(n)} &:= [J^{(a^{n-1})} f^{(n)}]^\top \text{ init} \\
 BP^{(k-1)} &:= BP^{(k)} [J^{(a^{k-2})} f^{(k-1)}]^\top \\
 [J^{(w^{k-2})} g]^\top &= BP^{(k-1)} [J^{(w^{k-2})} f^{(k-2)}]^\top
 \end{aligned}$$

- ⊙ why this is efficient ? ... what are the alternatives ?

- why backprop is efficient ? ... what are the alternatives ?
 - gradient by finite difference approximation

$$\frac{\partial g}{\partial w_i^{(k)}} \approx \frac{g(w_i^{(k)} + \epsilon) - g(w_i^{(k)})}{\epsilon}$$

$$\frac{\partial g}{\partial w_i^{(k)}} \approx \frac{g(w_i^{(k)} + \epsilon) - g(w_i^{(k)} - \epsilon)}{2\epsilon}$$

... Needs as many forward passes through the whole network as we have parameters.

- gradient by finite difference approximation
 - ... Needs as many forward passes through the whole network as we have parameters.



... LLama-7B: 7 Billion forward passes ... enjoy

- why backprop is efficient ? ... what are the alternatives ?
 - gradient by finite difference approximation ... Needs as many forward passes through the whole network as we have parameters. ... LLama-7B: 7 Billion forward passes ... enjoy
- Backprop needs 1 forward pass to get all layer feature maps $a^{(k)}$

$$\begin{aligned} [J^{(w^{k-1})} g]^\top &= BP^{(k)} [J^{(w^{k-1})} f^{(k-1)}]^\top \\ BP^{(k-1)} &:= BP^{(k)} [J^{(a^{k-2})} f^{(k-1)}]^\top \end{aligned}$$

- Backprop needs also for each layer:
 - one gradient computation for a single layer $f^{(k-1)}$ with respect to its direct weights w^{k-1}
 - one matrix multiplication to get $J^{(w^{k-1})} g$ for the parameters of a layer
 - one gradient computation for a single layer $f^{(k-1)}$ with respect to its direct inputs a^{k-2}
 - one matrix multiplication to get/update $BP^{(k-1)}$

What about the gradients for a single layer ?

For an affine layer they are trivial:

$$a^{(n)} = W^{(n)} a^{(n-1)} + b^{(n)}$$

$$a_j^{(n)} = W^{(n)}[j, :] a^{(n-1)} + b^{(n)}[j]$$

$$a_j^{(n)} = \sum_k W^{(n)}[j, k] a^{(n-1)}[k] + b^{(n)}[j]$$

$$\nabla^{W^{(n)}[j, :]} a_j^{(n)} = a^{(n-1)}$$

$$\nabla^{a^{(n-1)}} a_j^{(n)} = W^{(n)}[j, :]$$

obviously because

$$\frac{\partial}{\partial W^{(n)}[j, r]} \sum_k W^{(n)}[j, k] a^{(n-1)}[k] + b^{(n)}[j] = a^{(n-1)}[r]$$

$$\frac{\partial}{\partial a^{(n-1)}[r]} \sum_k W^{(n)}[j, k] a^{(n-1)}[k] + b^{(n)}[j] = W^{(n)}[j, r]$$

- why backprop is efficient ?
- Backprop needs 1 forward pass to get all layer feature maps $a^{(k)}$

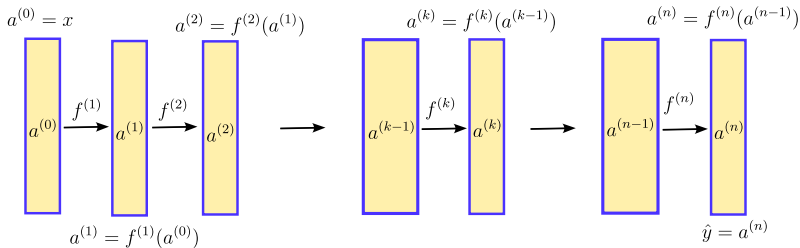
$$\begin{aligned} [J^{(w^{k-1})} g]^\top &= BP^{(k)} [J^{(w^{k-1})} f^{(k-1)}]^\top \\ BP^{(k-1)} &:= BP^{(k)} [J^{(a^{k-2})} f^{(k-1)}]^\top \end{aligned}$$

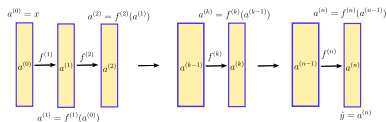
- Backprop needs also for each layer:
 - one gradient computation for a single layer $f^{(k-1)}$ with respect to its direct weights w^{k-1}
 - one matrix multiplication to get $J^{(w^{k-1})} g$ for the parameters of a layer
 - one gradient computation for a single layer $f^{(k-1)}$ with respect to its direct inputs a^{k-2}
 - one matrix multiplication to get/update $BP^{(k-1)}$
- the gradient computation steps are trivial for affine layers !

Why backprop is efficient ? Answer (II) via a simplified count:

- a neural network only composed of matrix multiplications $x \in \mathbb{R}^d$. All feature maps are of dimensionality d . All matrices are of shape (d, d) .

$$y = W^{(n)} W^{(n-1)} W^{(n-2)} \dots W^{(1)} x$$





A simplified count:

- a neural network only composed of matrix multiplications $x \in \mathbb{R}^d$. All feature maps are of dimensionality d . All matrices are of shape (d, d) .

$$y = W^{(n)} W^{(n-1)} W^{(n-2)} \dots W^{(1)} x$$

- finite difference:
 - one forward pass needs n matrix-vector multiplications $W^{(k)} a^{(k-1)}$ which are $O(d^2)$. Total: $O(nd^2)$
 - we have nd^2 parameters for applying finite difference. In total $O(n^2 d^4)$ Ops
- backprop:
 - obtaining the layer derivatives is trivial (for affine layers)
 - matrix multiplication is $O(d^3)$. we have 2 per layer. This is in total $O(nd^3)$ Ops

- ① Properties of Gradient Descent
- ② Stochastic gradient descent
- ③ Why gradient descent is hard in practice?
- ④ Compute the Gradient in PyTorch
- ⑤ Gradient descent in PyTorch
- ⑥ Chainrule via matrix multiplication and the Backpropagation algorithm
- ⑦ Einsum for generalized matrix-matrix multiplication type operations
- ⑧ Derivatives of Bilinear mappings

I have to implement:

$$T_{i,n,r,s} = \sum_{k,m,o} A_{i,k,m,n,o} B_{m,o,r} C_{k,m,r,s}$$

It might be slow and memory-consuming, but it is easy to be done!

It is a generalized matrix multiplication. Compare:

$$\begin{aligned} A, B &\mapsto (AB) \\ (AB)_{il} &= \sum_k A_{ik} B_{kl} \end{aligned}$$

`torch.matmul` <https://pytorch.org/docs/stable/generated/torch.matmul.html> works for the simple cases with broadcasting, but it is complicated in its rules.

Something easier ?

$$(AB)_{il} = \sum_k A_{ik} B_{kl}$$

```
ab= torch.einsum( 'ik,kl -> il' ,[a,b])
```

$$(Av)_i = \sum_k A_{ik} v_k$$

```
av= torch.einsum( 'ik,k -> i' ,[a,v])
```

$$w \cdot v = \sum_k w_k v_k$$

```
wdotv= torch.einsum( 'k,k -> ' ,[w,v])
```

a general way to do all kinds of batched and non-batched tensor multiplications:
`torch.einsum`

<https://rockt.github.io/2018/04/30/einsum>

4 rules:

- left of `— >`: all tensors separated by `,` which are to be multiplied and summed.
- right of `— >` the result tensor with remaining indices (maybe multiplied, but not summed over).
- **indices that have same name in multiple tensors, will get multiplied together** (must have same count in `.shape[i]` !)
- All **indices missing** right of `— >` **are summed out so that they are not present in the result.**

What are the einsum notations for ?

$$(a) \ x[:,], v[:,] \mapsto \sum_i x_i v_i$$

$$(b) \ A[:, :], v[:,] \mapsto \sum_i v_i A_{ik}$$

$$(c) \ v[:,], A[:, :], v[:,] \mapsto \sum_{i,k} v_i A_{ik} v_j$$

$$(d) \ A[:, :], C[:, :] \mapsto \sum_i A_{ik} C_{il}$$

$$(e) \ A[:, :], C[:, :] \mapsto \sum_i A_{ik} C_{ik}$$

Put your solutions in the [sli.do](#) link

What are the einsum notations for ?

$$(f) \ A[:, :], C[:, :] \mapsto \sum_{i,k} A_{ik} C_{ik}$$

$$(g) \ v[:,], A[:, :], B[:, :] \mapsto \sum_{i,k} v_i A_{ik} B_{kl}$$

$$(h) \ v[:,], A[:, :, :], B[:, :] \mapsto \sum_{k,l} v_l A_{ikl} B_{lr}$$

$$(i) \ A(5), B(3), C(4) \mapsto \sum_{k,m,o} A_{i,k,m,n,o} B_{m,o,r} C_{k,m,r,s}$$

Put your solutions in the [sli.do](#) link

- slower than built-in routines for standard matrix ops
- quantum computers ... indexing support for 3000-tensors ... Chinese writing as mandatory course ? :)

- ① Properties of Gradient Descent
- ② Stochastic gradient descent
- ③ Why gradient descent is hard in practice?
- ④ Compute the Gradient in PyTorch
- ⑤ Gradient descent in PyTorch
- ⑥ Chainrule via matrix multiplication and the Backpropagation algorithm
- ⑦ Einsum for generalized matrix-matrix multiplication type operations
- ⑧ Derivatives of Bilinear mappings

$f(\mathbf{X}, \mathbf{Y})$ is bilinear, if it is linear in argument $\mathbf{X}$ and also linear in argument $\mathbf{Y}$.

$$f(\mathbf{x}) = \mathbf{x}^\top A \mathbf{x}$$

$$\nabla f_k(\mathbf{x}) = ?$$

$$Df(\mathbf{x})[\mathbf{h}] = ?$$

Idea: get the directional derivatives for

$$f(A, B) = AB$$

$f(\mathbf{X}, \mathbf{Y})$ is bilinear, if it is linear in argument $\mathbf{X}$ and also linear in argument $\mathbf{Y}$.

Idea: get the directional derivatives for

$$f(A, B) = AB$$

$$\begin{aligned} f(A + tH, B + tK) &= f(A, B + tK) + f(tH, B + tK) = f(A, B) + tf(A, K) + tf(H, B + tK) \\ &= f(A, B) + tf(A, K) + tf(H, B) + t^2f(H, K) \end{aligned}$$

$$\begin{aligned} Df(A, B)[H, K] &= \lim_{t \rightarrow 0} \frac{f(A + tH, B + tK) - f(A, B)}{t} = \lim_{t \rightarrow 0} \frac{tf(A, K) + tf(H, B)}{t} + \frac{t^2f(H, K)}{t} \\ &= f(A, K) + f(H, B) + \lim_{t \rightarrow 0} \mathcal{O}(t) = f(A, K) + f(H, B) \end{aligned}$$

$f(\mathbf{X}, \mathbf{Y})$ is bilinear, if it is linear in argument $\mathbf{X}$ and also linear in argument $\mathbf{Y}$. Then:

$$Df(A, B)[H, K] = f(H, B) + f(A, K)$$

simple rule!

$f(\mathbf{X}, \mathbf{Y})$ is bilinear, if it is linear in argument $\mathbf{X}$ and also linear in argument $\mathbf{Y}$. Then:

$$Df(A, B)[H, K] = f(H, B) + f(A, K)$$

$$f(\mathbf{x}) = \mathbf{x}^\top A \mathbf{x}$$

Variant 1: take note that

$$g(\mathbf{x}, \mathbf{y}) = \mathbf{x}^\top A \mathbf{y}$$

is bilinear and

$$f(\mathbf{x}) = g(\mathbf{x}, \mathbf{x})$$

So:

$$Df(\mathbf{x})[\mathbf{h}] = Dg(x = \mathbf{x}, y = \mathbf{x})[H = \mathbf{h}, K = \mathbf{h}] = g(\mathbf{h}, \mathbf{x}) + g(\mathbf{x}, \mathbf{h}) = \mathbf{h}^\top A \mathbf{x} + \mathbf{x}^\top A \mathbf{h}$$

Done!!

Variant 2: use chainrule

$$f(\mathbf{x}) = \mathbf{x}^\top A \mathbf{x}$$
$$Df(\mathbf{x})[\mathbf{h}] = \text{chain rule!}$$

So:

$$f(\mathbf{x}) = \text{mul}(\mathbf{x}^\top, \text{mul}(A, \mathbf{x})) = \text{mul}(a(x), b(x))$$

$$a(\mathbf{x}) = \mathbf{x}^\top = \text{Id}^\top(\mathbf{x})$$

$$b(\mathbf{x}) = A\mathbf{x} = \text{mul}(A, \mathbf{x})$$

$$D\text{mul}(a(x), b(x))[h] = \text{mul}(Da(x)[h], b(x)) + \text{mul}(a(x), Db(x)[h])$$

$$\begin{aligned}f(\mathbf{x}) &= \mathbf{x}^\top A \mathbf{x} = \text{mul}(\mathbf{x}^\top, \text{mul}(A, \mathbf{x})) = \text{mul}(a(\mathbf{x}), b(\mathbf{x})) \\ D\text{mul}(a(\mathbf{x}), b(\mathbf{x}))[\mathbf{h}] &= \text{mul}(Da(\mathbf{x})[\mathbf{h}], b(\mathbf{x})) + \text{mul}(a(\mathbf{x}), Db(\mathbf{x})[\mathbf{h}])\end{aligned}$$

So:

$$\begin{aligned}\text{with } a(\mathbf{x}) &= \mathbf{x}^\top, \quad b(\mathbf{x}) = A\mathbf{x} \\ Df(\mathbf{x})[\mathbf{h}] &= \text{mul}(Da(\mathbf{x})[\mathbf{h}], b(\mathbf{x})) + \text{mul}(a(\mathbf{x}), Db(\mathbf{x})[\mathbf{h}]) \\ &= \text{mul}(\mathbf{h}^\top, b(\mathbf{x})) + \text{mul}(a(\mathbf{x}), A\mathbf{h}) \\ &= \text{mul}(\mathbf{h}^\top, A\mathbf{x}) + \text{mul}(\mathbf{x}^\top, A\mathbf{h}) \\ &= \mathbf{h}^\top A\mathbf{x} + \mathbf{x}^\top A\mathbf{h}\end{aligned}$$